



UNIVERSIDADE FEDERAL DE ALAGOAS
INSTITUTO DE COMPUTAÇÃO
CIÊNCIA DA COMPUTAÇÃO

Eduardo Maciel Alexandre - 23110441

Josenilton Ferreira da Silva Junior - 23110473

Lucas Cassiano Maciel dos Santos - 23110474

Maria Letícia Ventura

Relatório de Projeto AB2 - Tema 2
Compiladores

MACEIÓ, AL

2025

SUMÁRIO

1. INTRODUÇÃO.....	2
1.1 Contextualização.....	2
1.2 Objetivos.....	2
1.3 Características da Linguagem MiniPar 2025.1.....	3
1.4 Arquitetura e Metodologia.....	3
1.5 Links Externos.....	3
2. A LINGUAGEM MINIPAR 2025.1.....	4
2.1 Características Principais:.....	4
2.1.1 Paralelismo Explícito (PAR):.....	4
2.1.2 Execução Sequencial (SEQ):.....	4
2.1.3 Comunicação Exclusiva por Mensagens:.....	4
2.1.4 Sintaxe Semelhante ao Python:.....	4
2.1.5 Suporte à Orientação a Objetos (OO):.....	5
2.2 Estrutura Gramatical (BNF):.....	5
3. ANALISADOR LÉXICO.....	10
4. ANALISADOR SINTÁTICO.....	13
5. ANALISADOR SEMÂNTICO E TRATAMENTO DE ERROS.....	24
6. TABELA DE SÍMBOLOS.....	28
7. INTERFACE WEB.....	30
7.1 Endpoints.....	30
8. PRODUCT BACKLOG.....	31
9. SPRINTS BACKLOG.....	31
10. DIAGRAMA DE CASOS DE USO (UML).....	32
11. MODELAGEM DE COMPONENTES DE SOFTWARE (UML).....	32
12. DIAGRAMA DE CLASSES (UML).....	34
13. TESTES.....	35
a. Teste 1: Calculadora Cliente-Servidor.....	35
b. Teste 2: Execução Paralela: Fatorial e Fibonacci.....	36
c. Teste 3: Neurônio Simples (Perceptron).....	37
d. Teste 4: Rede Neural para XOR.....	38
e. Teste 5: Sistema de Recomendação E-commerce.....	38
f. Teste 6: Ordenação QuickSort.....	39
15 IMPLEMENTAÇÃO.....	39
15.1 Tecnologias Usadas.....	39
15.2 Dependências Principais.....	41
15.3 Ferramentas de Desenvolvimento.....	41
15.4 Justificativas Técnicas.....	41
15.5 Links Externos.....	42

1. INTRODUÇÃO

O desenvolvimento de compiladores e interpretadores representa um dos pilares fundamentais da Ciência da Computação, permitindo a tradução de linguagens de alto nível em instruções executáveis por computadores. Este documento apresenta o desenvolvimento de um **interpretador para a linguagem MiniPar 2025.1**, uma linguagem educacional orientada a objetos que suporta execução sequencial e paralela de instruções, além de comunicação entre processos via canais de comunicação.

1.1 Contextualização

Este projeto foi desenvolvido como requisito parcial da disciplina de **Compiladores (UFAL, 2025.1)**, ministrada pelo professor **Arturo Hernández Domínguez**, seguindo o **Tema 2 – MiniPar Orientado a Objetos**. A proposta visa não apenas a implementação de um interpretador funcional, mas também a aplicação de conceitos modernos de Engenharia de Software, especialmente **Componentes de Software** e **modelagem orientada a objetos utilizando UML**.

1.2 Objetivos

O objetivo principal deste trabalho é desenvolver um interpretador completo para a linguagem MiniPar 2025.1, contemplando:

- **Análise Léxica:** Reconhecimento de tokens através de expressões regulares
- **Análise Sintática:** Construção de uma Árvore Sintática Abstrata (AST) via parser descendente recursivo
- **Análise Semântica:** Verificação de tipos, escopos e validação de regras semânticas
- **Interpretação:** Execução direta do código através da AST
- **Suporte à Orientação a Objetos:** Classes, herança, instanciação e invocação de métodos
- **Execução Paralela:** Implementação de blocos PAR utilizando threads
- **Comunicação via Canais:** Implementação de primitivas `send` e `receive` através de sockets

1.3 Características da Linguagem MiniPar 2025.1

A linguagem MiniPar 2025.1 combina características de linguagens imperativas tradicionais com recursos de **paralelismo** e **orientação a objetos**. Entre suas principais características destacam-se:

- **Blocos de Execução:** **SEQ** (sequencial) e **PAR** (paralelo)
- **Tipos Primitivos:** **Bool**, **Int**, **String** e **c_channel** (canal de comunicação)
- **Estruturas de Controle:** **if/else**, **while**, **do**, **for**
- **Programação Orientada a Objetos:** Classes, herança, métodos e atributos
- **Comunicação entre Processos:** Operações **send** e **receive** via sockets
- **Funções e Recursão:** Suporte completo para definição e chamada de funções

1.4 Arquitetura e Metodologia

O projeto foi desenvolvido seguindo uma **arquitetura modular baseada em componentes**, onde cada fase do processo de compilação é representada por um componente independente e bem definido. A modelagem foi realizada utilizando **diagramas UML** (Casos de Uso, Componentes e Classes), seguindo boas práticas de Engenharia de Software.

A implementação foi realizada em **Java 17+**, adotando o paradigma orientado a objetos e princípios como clareza, coerência, evolução incremental e reprodutibilidade. O projeto também conta com:

- **Interface CLI:** Para execução via linha de comando
- **Interface Web:** Com syntax highlighting, exemplos prontos e execução em tempo real
- **Tratamento de Erros:** Exceções customizadas para erros léxicos, sintáticos e semânticos
- **Testes Abrangentes:** Seis programas de teste cobrindo diferentes aspectos da linguagem

1.5 Links Externos

Os [links externos](#) podem ser encontrados ao final deste relatório, contendo o acesso ao repositório do GitHub, ao documento base de referência e ao vídeo de demonstração.

2. A LINGUAGEM MINIPAR 2025.1

A Linguagem Minipar é uma linguagem de programação projetada para suportar a **execução sequencial (SEQ) e paralela (PAR)** de instruções, com Suporte a Paralelismo e Comunicação por Mensagens, que visa combinar a execução tradicional com o suporte nativo ao paralelismo e à comunicação entre processos.

2.1 Características Principais:

2.1.1 Paralelismo Explícito (**PAR**):

- A execução paralela é definida dentro do bloco de instruções **PAR**.
- O paralelismo é implementado utilizando **Threads**, tratadas como **Processos independentes** que **não compartilham informações**.

2.1.2 Execução Sequencial (**SEQ**):

- A execução tradicional (sequencial) de um conjunto de instruções é definida dentro do bloco **SEQ**.

2.1.3 Comunicação Exclusiva por Mensagens:

- A comunicação entre os processos (Threads independentes) é realizada **somente via mensagem**.
- Utiliza um tipo de variável dedicado: **c_channel** (communication channel).
- A comunicação é implementada através de um **Canal de Comunicação** (sugerido por *sockets* em Java ou Python).
- **Funções de Comunicação:**
 - **Send**: Envio de dados através do canal de comunicação (socket) para outro computador.
 - **Receive**: Recepção de dados de outro computador através do canal de comunicação.
 - Testes podem ser realizados no mesmo computador (**localhost**).

2.1.4 Sintaxe Semelhante ao Python:

- Os tipos de variáveis básicos são **Bool**, **Int** e **String**, seguindo a semântica do Python.

- As funções de Entrada/Saída (**Input** do teclado e **Output** na tela) também seguem o padrão Python.
- Comentários são indicados pelo símbolo **#**.
- Suporte à declaração e uso de **Funções**.

2.1.5 Suporte à Orientação a Objetos (OO):

- A linguagem inclui produções (regras BNF) para a sintaxe de Orientação a Objetos, como classes (**class**), herança (**extends**), declaração de métodos, instanciação (**new**) e chamada de métodos.
- A especificação inicial utiliza uma sintaxe similar a **Java** (ex: **<instanciacao>**), mas permite que o desenvolvedor utilize e represente a sintaxe de OO equivalente em **Python**, mantendo a flexibilidade.

2.2 Estrutura Gramatical (BNF):

A linguagem MiniPar é definida por uma Gramática BNF (Backus-Naur Form) que estabelece a seguinte estrutura:

```
# Gramática BNF da Linguagem MiniPar OOP - Tema 2
# Compiladores 2025.1 - Atividade 2

## PROGRAMA
<programa> ::= { <declaracao> }

<declaracao> ::= <declaracao_classe>
                | <declaracao_funcao>
                | <declaracao_variavel>
                | <comando>

## ORIENTAÇÃO A OBJETOS

<declaracao_classe> ::= "class" <identificador> [ "extends" <identificador> ]
                    "{" { <membro_classe> } "}"

<membro_classe> ::= <declaracao_variavel>
                  | <declaracao_metodo>

<declaracao_metodo> ::= <tipo_retorno> <identificador> "(" <parametros> ")"
                    "{" { <comando> } "}"

<instanciacao> ::= "new" <identificador> "(" [ <argumentos> ] ")"
```

```

<chamada_metodo> ::= <expressao> "." <identificador> "(" [ <argumentos> ] ")"

## FUNÇÕES

<declaracao_funcao> ::= "func" <identificador> "(" <parametros> ")"
                        "->" <tipo> "{" { <comando> } "}"

<parametros> ::= [ <parametro> { "," <parametro> } ]

<parametro> ::= <tipo> <identificador>

<argumentos> ::= <expressao> { "," <expressao> }

## VARIÁVEIS

<declaracao_variavel> ::= <tipo> <identificador> [ "=" <expressao> ]

<atribuicao> ::= <identificador> "=" <expressao>

## COMANDOS

<comando> ::= <declaracao_variavel>
            | <atribuicao>
            | <comando_if>
            | <comando_while>
            | <comando_do_while>
            | <comando_for>
            | <comando_return>
            | <comando_break>
            | <comando_continue>
            | <bloco_seq>
            | <bloco_par>
            | <comando_print>
            | <comando_input>
            | <chamada_metodo>
            | <expressao>

<comando_if> ::= "if" "(" <expressao> ")" "{" { <comando> } "}"
               [ "else" "{" { <comando> } "}" ]

<comando_while> ::= "while" "(" <expressao> ")" "{" { <comando> } "}"

<comando_for> ::= "for" "(" <tipo> <identificador> "in" <expressao> ")"
                 "{" { <comando> } "}"

<comando_do_while> ::= "do" "{" { <comando> } "}" "while" "(" <expressao> ")"
                      ";"

<comando_print> ::= "print" "(" [ <argumentos> ] ")" ";"

<comando_input> ::= "input" "(" [ <expressao> ] ")"

```

```

<comando_return> ::= "return" [ <expressao> ]

<comando_break> ::= "break"

<comando_continue> ::= "continue"

<bloco_seq> ::= "seq" "{" { <comando> } "}"

<bloco_par> ::= "par" "{" { <comando> } "}"

## CANAIS E MENSAGENS

<declaracao_canal> ::= "c_channel" "(" <identificador> { "," <identificador> }
                    ")" ";"
                    | "c_channel" <identificador> <identificador>
<identificador> ";"

<comando_send> ::= <identificador> "." "send" "(" <argumentos> ")" ";"

<comando_receive> ::= <identificador> "." "receive" "(" <argumentos> ")" ";"

## EXPRESSÕES

<expressao> ::= <atribuicao>
              | <expr_logica_ou>

<expr_logica_ou> ::= <expr_logica_e> { "||" <expr_logica_e> }

<expr_logica_e> ::= <expr_igualdade> { "&&" <expr_igualdade> }

<expr_igualdade> ::= <expr_relacional> { ( "==" | "!=" ) <expr_relacional> }

<expr_relacional> ::= <expr_aditiva> { ( ">" | ">=" | "<" | "<=" )
<expr_aditiva> }

<expr_aditiva> ::= <expr_multiplicativa> { ( "+" | "-" ) <expr_multiplicativa> }

<expr_multiplicativa> ::= <expr_unaria> { ( "*" | "/" | "%" ) <expr_unaria> }

<expr_unaria> ::= ( "!" | "-" ) <expr_unaria>
                | <expr_chamada>

<expr_chamada> ::= <expr_primaria>
                { "(" [ <argumentos> ] ")" | "." <identificador> | "["
<expressao> "]" }

<expr_primaria> ::= <numero>
                  | <string>
                  | "true"
                  | "false"

```



```

| <identificador>
| <instanciacao>
| <literal_lista>
| <literal_dict>
| "(" <expressao> ")"

<literal_lista> ::= "[" [ <expressao> { "," <expressao> } ] "]"

<literal_dict> ::= "{" [ <expressao> ":" <expressao> { "," <expressao> ":"
<expressao> } ] "}"

## TIPOS

<tipo> ::= "number"
| "string"
| "bool"
| "void"
| "list"
| "dict"
| <identificador> # tipo customizado (classe)

<tipo_retorno> ::= <tipo>

## LÉXICO

<identificador> ::= <letra> { <letra> | <digito> | "_" }

<numero> ::= <digito> { <digito> } [ "." <digito> { <digito> } ]

<string> ::= "'" { <caractere> } "'"

<letra> ::= "A" | "B" | ... | "Z" | "a" | "b" | ... | "z"

<digito> ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"

<caractere> ::= qualquer caractere ASCII exceto "'"

## COMENTÁRIOS

<comentario_linha> ::= "#" { <qualquer_caractere> } <nova_linha>

<comentario_bloco> ::= "/*" { <qualquer_caractere> } "*/"

## PALAVRAS-RESERVADAS

class, extends, new, this, super,
func, var, if, else, while, do, for, return, break, continue,
seq, par, c_channel, s_channel, in, print, input,
number, string, bool, void, list, dict,
true, false
## OPERADORES

```

Aritméticos: +, -, *, /, %
Relacionais: ==, !=, <, <=, >, >=
Lógicos: &&, ||, !
Atribuição: =
Acesso: .
Outros: ->, :, ,, ;, (,), {, }, [,]

3. ANALISADOR LÉXICO

O Analisador Léxico (Lexer) tem como objetivo transformar o fluxo de caracteres em unidades sintaticamente relevantes, como identificadores, literais, operadores e as palavras-chave específicas da MiniPar, como `SEQ`, `PAR`, `class`, `chan`, e os tipos de dados `Int`, `String`, `c_channel`, etc.

O Lexer é responsável por tarefas essenciais, como: **Contagem de Linhas**, **Tratamento de Comentários** e **Reconhecimento de Palavras-Chave**, atuando como o **componente de entrada** que prepara o código-fonte para ser processado pelo Analisador Sintático, garantindo que apenas *tokens* válidos cheguem à próxima etapa do interpretador. No projeto utiliza-se uma abordagem **Processual e Iterativa**, com forte foco na leitura sequencial do código-fonte, caractere por caractere, para agrupar em Tokens.

Abaixo segue o exemplo de pseudocódigo do Analisador Léxico no projeto:

```
# Objetivo: transformar o código-fonte em uma sequência de tokens com
linha/coluna
# Suporta comentários iniciados por # até o fim da linha

enum TokenKind {
    IDENT, INT_LIT, STRING_LIT, BOOL_LIT,
    CLASS, EXTENDS, SEQ, PAR, IF, ELSE, WHILE, RETURN, NEW,
    VOID, INT, BOOL, STRING, C_CHANNEL, SEND, RECEIVE, PRINT, INPUT,
    TRUE, FALSE, PROGRAMA_MINIPAR,
    PLUS, MINUS, STAR, SLASH, MOD,
    ASSIGN, EQ, NEQ, LT, LTE, GT, GTE, AND, OR, NOT,
    DOT, COMMA, SEMI, COLON, LPAREN, RPAREN, LBRACE, RBRACE,
    EOF
}

KEYWORDS = {
    "programa-minipar": PROGRAMA_MINIPAR,
    "class": CLASS, "extends": EXTENDS, "SEQ": SEQ, "PAR": PAR,
    "if": IF, "else": ELSE, "while": WHILE, "return": RETURN,
    "new": NEW, "void": VOID, "int": INT, "bool": BOOL, "string": STRING,
    "c_channel": C_CHANNEL, "send": SEND, "receive": RECEIVE,
    "print": PRINT, "input": INPUT,
    "true": TRUE, "false": FALSE
}

struct Token { kind: TokenKind, lexeme: string, line: int, col: int }

function lex(input: string) -> List<Token>:
```

```

tokens = []
i = 0; line = 1; col = 1

function peek(k=0): return input[i+k] if i+k < len(input) else '\0'
function advance():
    ch = input[i]; i = i + 1
    if ch == '\n': line = line + 1; col = 1
    else: col = col + 1
    return ch
function add(kind, text): tokens.append(Token(kind, text, line, col))

while i < len(input):
    ch = peek()

    # Espaços e quebras
    if ch in [' ', '\t', '\r']:
        advance(); continue
    if ch == '\n':
        advance(); continue

    # Comentário: # até fim da linha
    if ch == '#':
        while i < len(input) and peek() != '\n':
            advance()
        continue

    # Identificadores e palavras-chave
    if isLetter(ch) or ch == '_':
        startLine = line; startCol = col
        buf = ""
        while isLetterOrDigit(peek()) or peek() == '_':
            buf += advance()
        kind = KEYWORDS.get(buf, IDENT)
        tokens.append(Token(kind, buf, startLine, startCol))
        continue

    # Números inteiros
    if isDigit(ch):
        startLine = line; startCol = col
        buf = ""
        while isDigit(peek()):
            buf += advance()
        tokens.append(Token(INT_LIT, buf, startLine, startCol))
        continue

    # Strings "..."
    if ch == '"':
        startLine = line; startCol = col
        advance() # consume "
        buf = ""
        while i < len(input) and peek() != '"':

```

```

        if peek() == '\\': # escape simples
            advance(); buf += translateEscape(advance())
        else:
            buf += advance()
    if peek() != '':
        error("String não terminada", startLine, startCol)
    else:
        advance() # fecha "
        tokens.append(Token(String_LIT, buf, startLine, startCol))
    continue

# Operadores de 2 chars
two = peek() + peek(1)
if two in ["==", "!=", "<=", ">=", "&&", "||"]:
    startLine = line; startCol = col
    advance(); advance()
    tokens.append(Token(mapTwoChar(two), two, startLine, startCol))
    continue

# Operadores/símbolos de 1 char
map1 = {
    '+': PLUS, '-': MINUS, '*': STAR, '/': SLASH, '%': MOD,
    '=': ASSIGN, '<': LT, '>': GT, '!': NOT,
    '.': DOT, ',': COMMA, ';': SEMI, ':': COLON,
    '(': LPAREN, ')': RPAREN, '{': LBRACE, '}': RBRACE
}
i ch in map1:
    startLine = line; startCol = col
    kind = map1[ch]; advance()
    tokens.append(Token(kind, ch, startLine, startCol))
    continue

# Caractere inválido
error("Caractere inválido: " + ch, line, col)
advance()

tokens.append(Token EOF, "", line, col))
return tokens

```

4. ANALISADOR SINTÁTICO

O Analisador Sintático (Parser) tem como função consumir os *tokens* gerados pelo Analisador Léxico (Lexer), verificando se a estrutura do código-fonte obedece à Gramática BNF da MiniPar. Cada não-terminal da gramática (como `analisar_programa_minipar` ou `analisar_classe`) é implementado como um procedimento que invoca outros procedimentos (recursão) e utiliza a função `casar_token()` para conferir e consumir terminais (palavras-chave e símbolos). No projeto utiliza-se o modelo de **Análise Preditiva Descendente Recursiva**.

O Parser tem a responsabilidade crucial de lidar com os comandos sequenciais (**SEQ**) e paralelos (**PAR**), e especialmente com a sintaxe de **Orientação a Objetos** (classes, métodos, instanciação e chamadas de método). O resultado da análise sintática é a construção da **Árvore Sintática (AST)**, que será utilizada posteriormente pelo Analisador Semântico e pelo componente de Execução do Interpretador.

Abaixo segue o exemplo de pseudocódigo do Analisador Sintático no projeto:

```
# Objetivo: construir AST a partir dos tokens, com recuperação simples de erros.
# Estratégia: descendente recursivo; expressões com precedência (Pratt ou climbing).

# Nós AST (amostra):
# Program(classes: [ClassDecl], body: Block?)
# ClassDecl(name, superName?, attrs: [AttrDecl], methods: [MethodDecl])
# AttrDecl(type, name)
# MethodDecl(retType, name, params: [Param], body: Block)
# Block(kind: 'SEQ'|'PAR'|'{', stmts: [Stmt])
# Stmt: Assign, If, While, Return, Print, ExprStmt, ChannelDecl, Send, Receive, VarDecl
# Expr: Binary, Unary, Literal, Var, Call(obj,name,args), New(class,args), Get(obj, name)

struct Parser { tokens: List<Token>, i: int, errors: List<ParseError> }

function parse(tokens) -> Program:
    p = Parser(tokens)
    classes = []
    while p.match(CLASS):
        classes.append(p.parseClassDecl())
    body = null
    if p.peekKind() in [SEQ, PAR, LBRACE, IF, WHILE, IDENT, PRINT, C_CHANNEL, RETURN]:
        body = p.parseTopLevelBlockOrStmts()
    p.consume(EOF, "Esperado fim do arquivo")
```

```

    return Program(classes, body)

# Utilidades
method match(kind): if peekKind()==kind { advance(); return true } else false
method consume(kind, msg): if match(kind) return prev else errorSync(msg)
method check(kind): return peekKind()==kind
method advance(): i = i+1; return tokens[i-1]
method peekKind(): return tokens[i].kind
method prev(): return tokens[i-1]
method syncTo(setOfKinds): while not at end and peekKind() not in setOfKinds:
    advance()

# Declarações de classe
method parseClassDecl() -> ClassDecl:
    name = consume(IDENT, "Nome da classe esperado")
    superName = null
    if match(EXTENDS):
        superName = consume(IDENT, "Nome da superclasse esperado")
    consume(LBRACE, "'{' esperado")
    attrs = []
    methods = []
    while not check(RBRACE) and not check EOF:
        if isTypeStart(peekKind()):
            # atributo ou assinatura de método
            save = i
            t = parseType()
            id = consume(IDENT, "Identificador esperado")
            if check(LPAREN):
                # método
                i = save # volta e re-parseia completando método
                methods.append(parseMethodDecl())
            else:
                # atributo
                consume(SEMI, "';' esperado após atributo")
                attrs.append(AttrDecl(t, id.lexeme))
            else:
                methods.append(parseMethodDecl())
    consume(RBRACE, "'}' esperado")
    return ClassDecl(name.lexeme, superName?.lexeme, attrs, methods)

method parseMethodDecl() -> MethodDecl:
    ret = parseTypeOrVoid()
    name = consume(IDENT, "Nome do método esperado")
    consume(LPAREN, "'(' esperado")
    params = []
    if not check(RPAREN):
        do:
            t = parseType()
            pid = consume(IDENT, "Nome de parâmetro esperado")
            params.append(Param(t, pid.lexeme))
        while match(COMMA)

```

```

consume(RPAREN, "'')' esperado")
body = parseBlockLike()
return MethodDecl(ret, name.lexeme, params, body)

# Blocos (SEQ, PAR ou { ... })
method parseTopLevelBlockOrStmts() -> Block:
    return parseBlockLike()

method parseBlockLike() -> Block:
    if match(SEQ):
        stmts = parseStmtList()
        return Block('SEQ', stmts)
    if match(PAR):
        stmts = parseStmtList()
        return Block('PAR', stmts)
    if match(LBRACE):
        stmts = []
        while not check(RBRACE) and not check EOF):
            stmts.append(parseStmt())
            consume(RBRACE, "'})' esperado")
        return Block('{', stmts)
    # fallback: bloco implícito com 1 ou mais stmts até sincronização
    stmts = []
    do: stmts.append(parseStmt())
    while not check(RBRACE) and not check EOF) and not check(SEQ) and not
check(PAR)
    return Block('{', stmts)

method parseStmtList() -> List[Stmt]:
    # Aceita sequência de instruções até um marcador de fim (RBRACE, EOF,
palavra-chave de topo)
    stmts = []
    while not check(RBRACE) and not check EOF) and not check(SEQ) and not
check(PAR):
        stmts.append(parseStmt())
    return stmts

# Instruções
method parseStmt() -> Stmt:
    if match(IF): return parseIf()
    if match(WHILE): return parseWhile()
    if match(RETURN):
        expr = null
        if not check(SEMI) and not check(RBRACE):
            expr = parseExpr()
        optional(SEMI)
        return Return(expr)
    if match(PRINT):
        consume(LPAREN, "'(' esperado")
        e = parseExpr()
        consume(RPAREN, "')' esperado")

```



```

    optional(SEMI)
    return Print(e)
if match(C_CHANNEL):
    # c_channel name comp1 comp2
    name = consume(IDENT, "Nome do canal esperado")
    c1 = consume(IDENT, "Computador 1 esperado")
    c2 = consume(IDENT, "Computador 2 esperado")
    optional(SEMI)
    return ChannelDecl(name.lexeme, c1.lexeme, c2.lexeme)
# Decl local: tipo id (= expr)?
if isTypeStart(peekKind()):
    t = parseType()
    id = consume(IDENT, "Identificador esperado")
    init = null
    if match(ASSIGN):
        init = parseExpr()
    optional(SEMI)
    return VarDecl(t, id.lexeme, init)
# Expressão (pode ser atribuição, chamada, send/receive, etc.)
e = parseExpr()
optional(SEMI)
return ExprStmt(e)

method parseIf() -> Stmt:
    consume(LPAREN, "'(' esperado")
    cond = parseExpr()
    consume(RPAREN, "')' esperado")
    thenB = parseBlockLike()
    elseB = null
    if match(ELSE):
        elseB = parseBlockLike()
    return If(cond, thenB, elseB)

method parseWhile() -> Stmt:
    consume(LPAREN, "'(' esperado")
    cond = parseExpr()
    consume(RPAREN, "')' esperado")
    body = parseBlockLike()
    return While(cond, body)

# Tipos
method parseTypeOrVoid():
    if match(VOID): return Type.Void
    return parseType()

method parseType():
    if match(INT): return Type.Int
    if match(BOOL): return Type.Bool
    if match(STRING): return Type.String
    if match(C_CHANNEL): return Type.Channel
    # tipo de classe

```

```

id = consume(IDENT, "Tipo esperado")
return Type.Class(id.lexeme)

function isTypeStart(k):
    return k in [INT, BOOL, STRING, C_CHANNEL, IDENT]

# Expressões (precedência: ||, &&, ==/!=, </<=/>=, +/-, */%, unários, primários)
method parseExpr(): return parseOr()

method parseOr():
    e = parseAnd()
    while match(OR):
        op = prev(); rhs = parseAnd()
        e = Binary(e, op, rhs)
    return e

method parseAnd():
    e = parseEquality()
    while match(AND):
        op = prev(); rhs = parseEquality()
        e = Binary(e, op, rhs)
    return e

method parseEquality():
    e = parseRel()
    while match(EQ) or match(NEQ):
        op = prev(); rhs = parseRel()
        e = Binary(e, op, rhs)
    return e

method parseRel():
    e = parseAdd()
    while match(LT) or match(LTE) or match(GT) or match(GTE):
        op = prev(); rhs = parseAdd()
        e = Binary(e, op, rhs)
    return e

method parseAdd():
    e = parseMul()
    while match(PLUS) or match(MINUS):
        op = prev(); rhs = parseMul()
        e = Binary(e, op, rhs)
    return e

method parseMul():
    e = parseUnary()
    while match(STAR) or match(SLASH) or match(MOD):
        op = prev(); rhs = parseUnary()
        e = Binary(e, op, rhs)
    return e

```

```

method parseUnary():
    if match(NOT) or match(MINUS):
        op = prev(); rhs = parseUnary()
        return Unary(op, rhs)
    return parsePostfix()

# Pós-fixos: acesso ., chamada, atribuição simples
method parsePostfix():
    e = parsePrimary()
    loop:
        if match(DOT):
            name = consume(IDENT, "Membro esperado após '.'")
            if match(LPAREN): # chamada método: obj.name(...)
                args = parseArgList()
                e = Call(e, name.lexeme, args)
            else:
                e = Get(e, name.lexeme)
            continue
        # atribuição: id = expr (somente se o LHS é Var ou Get)
        if match(ASSIGN):
            rhs = parseExpr()
            return Assign(e, rhs)
        break
    return e

method parseArgList():
    args = []
    if not check(RPAREN):
        do: args.append(parseExpr()) while match(COMMA)
    consume(RPAREN, "')' esperado")
    return args

method parsePrimary():
    if match(INT_LIT): return Literal(int(prev().lexeme))
    if match(STRING_LIT): return Literal(prev().lexeme)
    if match(TRUE): return Literal(true)
    if match(FALSE): return Literal(false)
    if match(IDENT):
        return Var(prev().lexeme)
    if match(NEW):
        cname = consume(IDENT, "Classe esperada após 'new'")
        consume(LPAREN, "'(' esperado")
        args = []
        if not check(RPAREN):
            do: args.append(parseExpr()) while match(COMMA)
        consume(RPAREN, "')' esperado")
        return New(cname.lexeme, args)
    if match(LPAREN):
        e = parseExpr(); consume(RPAREN, "')' esperado"); return e
    error("Expressão inválida", peek().line, peek().col)
    return Literal(null)

```

```
# Recuperação de erros básica: sincroniza em ; ou } ou palavra-chave de início de stmt
method errorSync(msg):
    reportError(msg, peek().line, peek().col)
    syncTo({SEMI, RBRACE, IF, WHILE, RETURN, PRINT, C_CHANNEL, SEQ, PAR, CLASS})
    if match(SEMI): return prev()
    return prev()
```

```
CLASSE Parser
    // Campos/Atributos
    LEXER: Lexer // Uma instância do analisador léxico
    TOKEN_ATUAL: Token // O token atualmente em foco

    // Construtor
    CONSTRUTOR(lexer)
        ESTE.LEXER = lexer
        CHAMAR ESTE.avancar_token() // Inicializa TOKEN_ATUAL
    FIM CONSTRUTOR

    // -----
    // MÉTODOS DE CONTROLE E ERRO
    // -----

    // Avança para o próximo token fornecido pelo lexer
    METODO avancar_token()
        ESTE.TOKEN_ATUAL = ESTE.LEXER.proximo_token()
    FIM METODO

    // Verifica se o token atual corresponde ao tipo esperado
    // Se sim, avança para o próximo token. Se não, lança um erro.
    METODO consumir(tipo_esperado)
        SE ESTE.TOKEN_ATUAL.TIPO_DO_TOKEN FOR IGUAL A tipo_esperado
            TOKEN_CONSUMIDO = ESTE.TOKEN_ATUAL
            CHAMAR ESTE.avancar_token()
            RETORNA TOKEN_CONSUMIDO
        SENAÓ
            CHAMAR ESTE.relatar_erro_sintatico(tipo_esperado)
            // Mecanismo de recuperação de erro pode ser implementado aqui
            RETORNA NULO // Em caso de erro
        FIM SE
    FIM METODO

    // Lida com erros de sintaxe
    METODO relatar_erro_sintatico(tipo_esperado)
        MENSAGEM_ERRO = "Erro de Sintaxe: Esperado " + tipo_esperado + " mas encontrou " + ESTE.TOKEN_ATUAL.TIPO_DO_TOKEN + " ('" + ESTE.TOKEN_ATUAL.LEXEMA + "')"
        IMPRIMIR_ERRO(MENSAGEM_ERRO + " na linha " + ESTE.TOKEN_ATUAL.LINHA)
```

```

    // Em um interpretador real, você pode lançar uma exceção
FIM METODO

// -----
// MÉTODOS DE ANÁLISE SINTÁTICA (Regras de Gramática)
// -----

// Regra Principal: Programa -> (Declaração | Função)* EOF
METODO analisar_programa()
    NO_PROGRAMA = NOVO NoBloco() // O programa inteiro é um grande bloco
    ENQUANTO ESTE.TOKEN_ATUAL.TIPO_DO_TOKEN NÃO FOR EOF
        DECLARACAO = CHAMAR ESTE.analisar_declaracao()
        SE DECLARACAO NÃO FOR NULO
            ADICIONAR(NO_PROGRAMA.DECLARACOES, DECLARACAO)
        FIM SE
        // Mecanismo para evitar loops infinitos em caso de erro léxico
    FIM ENQUANTO
    RETORNA NO_PROGRAMA
FIM METODO

// Regra: Declaração -> DeclVariavel | DeclIf | DeclWhile | Expressao ;
METODO analisar_declaracao()
    ESCOLHA ESTE.TOKEN_ATUAL.TIPO_DO_TOKEN
        CASO PALAVRA_CHAVE_VAR:
            RETORNA CHAMAR ESTE.analisar_declaracao_variavel()
        CASO PALAVRA_CHAVE_SE:
            RETORNA CHAMAR ESTE.analisar_declaracao_se()
        CASO PALAVRA_CHAVE_ENQUANTO:
            RETORNA CHAMAR ESTE.analisar_declaracao_enquanto()
        CASO OUTRO:
            // Se não for uma declaração formal, assume-se que é uma
            expressão seguida de ponto e vírgula
            NO_EXPRESSAO = CHAMAR ESTE.analisar_expressao()
            CHAMAR ESTE.consumir(PONTO_VIRGULA)
            RETORNA NO_EXPRESSAO
    FIM ESCOLHA
FIM METODO

// Regra: DeclVariavel -> 'var' IDENTIFICADOR [ '=' Expressao ] ';'
METODO analisar_declaracao_variavel()
    CHAMAR ESTE.consumir(PALAVRA_CHAVE_VAR)
    NOME = CHAMAR ESTE.consumir(IDENTIFICADOR)

    NO_DECL = NOVO NoDeclaracaoVariavel()
    NO_DECL.NOME_VARIABEL = NOME
    NO_DECL.VALOR_INICIAL = NULO // Inicialmente nulo

    SE ESTE.TOKEN_ATUAL.TIPO_DO_TOKEN FOR IGUAL A SINAL_ATRIBUICAO
        CHAMAR ESTE.avancar_token() // Consome '='
        NO_DECL.VALOR_INICIAL = CHAMAR ESTE.analisar_expressao()
    FIM SE

```

```

        CHAMAR ESTE.consumir(PONTO_VIRGULA)
        RETORNA NO_DECL
    FIM METODO

// -----
// MÉTODOS PARA EXPRESSÕES (Precedência por Métodos)
// -----

// Início da cadeia de precedência: Expressao -> ExpressaoLogica
METODO analisar_expressao()
    RETORNA CHAMAR ESTE.analisar_expressao_logica()
FIM METODO

// Regra: ExpressaoLogica -> ExpressaoIgualdade ( ('&&' | '||')
ExpressaoIgualdade )*
METODO analisar_expressao_logica()
    NO_ESQUERDA = CHAMAR ESTE.analisar_expressao_igualdade()

    ENQUANTO ESTE.TOKEN_ATUAL.TIPO_DO_TOKEN FOR OP_AND OU
ESTE.TOKEN_ATUAL.TIPO_DO_TOKEN FOR OP_OR
        OPERADOR = CHAMAR ESTE.avancar_token()
        NO_DIREITA = CHAMAR ESTE.analisar_expressao_igualdade()
        NO_ESQUERDA = NOVO NoExpressaoBinaria(OPERADOR, NO_ESQUERDA,
NO_DIREITA)
    FIM ENQUANTO

    RETORNA NO_ESQUERDA
FIM METODO

// Regra: ExpressaoIgualdade -> ExpressaoAditiva ( ('==' | '!=')
ExpressaoAditiva )*
METODO analisar_expressao_igualdade()
    NO_ESQUERDA = CHAMAR ESTE.analisar_expressao_aditiva()

    ENQUANTO ESTE.TOKEN_ATUAL.TIPO_DO_TOKEN FOR OP_IGUALDADE OU
ESTE.TOKEN_ATUAL.TIPO_DO_TOKEN FOR OP_DIFERENCA
        OPERADOR = CHAMAR ESTE.avancar_token()
        NO_DIREITA = CHAMAR ESTE.analisar_expressao_aditiva()
        NO_ESQUERDA = NOVO NoExpressaoBinaria(OPERADOR, NO_ESQUERDA,
NO_DIREITA)
    FIM ENQUANTO

    RETORNA NO_ESQUERDA
FIM METODO

// Regra: ExpressaoAditiva -> ExpressaoMultiplicativa ( ('+' | '-')
ExpressaoMultiplicativa )*
METODO analisar_expressao_aditiva()
    NO_ESQUERDA = CHAMAR ESTE.analisar_expressao_multiplicativa()

```

```

        ENQUANTO ESTE.TOKEN_ATUAL.TIPO_DO_TOKEN FOR SINAL MAIS OU
        ESTE.TOKEN_ATUAL.TIPO_DO_TOKEN FOR SINAL MENOS
            OPERADOR = CHAMAR ESTE.avancar_token()
            NO_DIREITA = CHAMAR ESTE.analisar_expressao_multiplicativa()
            NO_ESQUERDA = NOVO NoExpressaoBinaria(OPERADOR, NO_ESQUERDA,
NO_DIREITA)
        FIM ENQUANTO

        RETORNA NO_ESQUERDA
    FIM METODO

    // Regra: ExpressaoMultiplicativa -> ExpressaoUnaria ( ('*' | '/')
ExpressaoUnaria )*
    METODO analisar_expressao_multiplicativa()
        NO_ESQUERDA = CHAMAR ESTE.analisar_expressao_unaria()

        ENQUANTO ESTE.TOKEN_ATUAL.TIPO_DO_TOKEN FOR SINAL MULTIPLICACAO OU
        ESTE.TOKEN_ATUAL.TIPO_DO_TOKEN FOR SINAL DIVISAO
            OPERADOR = CHAMAR ESTE.avancar_token()
            NO_DIREITA = CHAMAR ESTE.analisar_expressao_unaria()
            NO_ESQUERDA = NOVO NoExpressaoBinaria(OPERADOR, NO_ESQUERDA,
NO_DIREITA)
        FIM ENQUANTO

        RETORNA NO_ESQUERDA
    FIM METODO

    // Regra: ExpressaoUnaria -> ('-' | '!') ExpressaoUnaria | ExpressaoPrimaria
    METODO analisar_expressao_unaria()
        SE ESTE.TOKEN_ATUAL.TIPO_DO_TOKEN FOR SINAL MENOS OU
        ESTE.TOKEN_ATUAL.TIPO_DO_TOKEN FOR SINAL NEGACAO
            OPERADOR = CHAMAR ESTE.avancar_token()
            OPERANDO = CHAMAR ESTE.analisar_expressao_unaria()
            RETORNA NOVO NoExpressaoUnaria(OPERADOR, OPERANDO)
        SENA
            RETORNA CHAMAR ESTE.analisar_expressao_primaria()
        FIM SE
    FIM METODO

    // Regra: ExpressaoPrimaria -> LITERAL | IDENTIFICADOR | '(' Expressao ')' |
ChamadaFuncao
    METODO analisar_expressao_primaria()
        ESCOLHA ESTE.TOKEN_ATUAL.TIPO_DO_TOKEN
            CASO NUMERO_INTEIRO:
            CASO STRING_LITERAL:
            CASO PALAVRA_CHAVE_VERDADEIRO:
            CASO PALAVRA_CHAVE_FALSO:
                // Cria nó para literal e avança
                TOKEN_LITERAL = CHAMAR ESTE.avancar_token()
                RETORNA NOVO NoLiteral(TOKEN_LITERAL)

```

```

        CASO IDENTIFICADOR:
            // Checa se é uma chamada de função ou apenas uma variável
            SE CHAMAR ESTE.espiar_proximo_caractere() FOR IGUAL A
PARENTESE_ABRE
            RETORNA CHAMAR ESTE.analisar_chamada_funcao()
        SENAO
            RETORNA NOVO NoVariavel(CHAMAR ESTE.avancar_token())
        FIM SE

        CASO PARENTESE_ABRE:
            CHAMAR ESTE.avancar_token() // Consome '('
            EXPRESSAO = CHAMAR ESTE.analisar_expressao()
            CHAMAR ESTE.consumir(PARENTESE_FECHA) // Espera ')'
            RETORNA EXPRESSAO

        CASO OUTRO:
            CHAMAR ESTE.relatar_erro_sintatico("Literal, Identificador ou
'('")
            CHAMAR ESTE.avancar_token() // Tentativa de avanço para
recuperação
            RETORNA NULO
        FIM ESCOLHA
    FIM METODO

    // Implementação de outros métodos como analisar_declaracao_se(),
    analisar_chamada_funcao(), etc.
    // ...

FIM CLASSE

```


5. ANALISADOR SEMÂNTICO E TRATAMENTO DE ERROS

O Analisador Semântico é a fase do interpretador responsável por garantir que o programa obedeça às regras de significado da Linguagem MiniPar 2025.1. Sua função primária é verificar a coerência de tipos nas expressões e atribuições, além de validar a correta utilização de identificadores em seus respectivos escopos. No contexto da Orientação a Objetos e da comunicação paralela, ele valida a herança de classes e o uso apropriado das operações **Send** e **Receive** no **c_channel**.

O tratamento de erros é uma exigência essencial, conforme a Observação 5 do projeto, e não se limita às falhas léxicas ou sintáticas. O Analisador Semântico é o ponto onde são detectados erros de lógica do programa, como incompatibilidade de tipos ou o uso de variáveis não declaradas. Ele também deve assegurar que as regras de precedência dos operadores aritméticos (+, -, *, /) sejam respeitadas, conforme a Observação 2, evitando ambiguidades e garantindo a correta interpretação matemática das expressões.

Para realizar todas essas verificações, o Analisador Semântico colabora diretamente com a **Tabela de Símbolos**, que armazena os metadados contextuais do código, como tipo e escopo dos identificadores. Ao consultar esta tabela, o Semântico pode verificar a validade das operações e garantir que as chamadas de métodos e as declarações de canais de comunicação estejam em conformidade com as definições da linguagem, fundamentando a robustez do interpretador.

Exemplo de pseudocódigo de um Analisador Semântico em um Interpretador orientado a objetos:

```
CLASSE AnalisadorSemantico
  // Campos/Atributos
  TABELA_SIMBOLOS: TabelaDeSimbolos
  AST_RAIZ: NoAST // A raiz da Árvore Sintática Abstrata
  TEM_ERROS: Booleano = FALSO

  // Construtor
  CONSTRUTOR(ast_raiz)
    ESTE.AST_RAIZ = ast_raiz
    ESTE.TABELA_SIMBOLOS = NOVA TabelaDeSimbolos()
  FIM CONSTRUTOR

  // -----
```

```

// MÉTODO PRINCIPAL
// -----

METODO analisar()
    IMPRIMIR("Iniciando Análise Semântica...")
    CHAMAR ESTE.visitar(ESTE.AST_RAIZ)

    SE ESTE.TEM_ERROS
        IMPRIMIR_ERRO("Análise Semântica falhou com erros.")
        RETORNA FALSO
    SENA
        IMPRIMIR("Análise Semântica concluída sem erros.")
        RETORNA VERDADEIRO
    FIM SE
FIM METODO

// -----
// VISITORS (Percurso da AST)
// -----

// O método de despacho principal, que chama o visitor apropriado
METODO visitar(no)
    SE no É INSTÂNCIA DE NoBloco
        CHAMAR ESTE.visitar_bloco(no)
    SENA SE no É INSTÂNCIA DE NoDeclaracaoVariavel
        CHAMAR ESTE.visitar_declaracao_variavel(no)
    SENA SE no É INSTÂNCIA DE NoAtribuicao
        CHAMAR ESTE.visitar_atribuicao(no)
    SENA SE no É INSTÂNCIA DE NoExpressaoBinaria
        CHAMAR ESTE.visitar_expressao_binaria(no)
    SENA SE no É INSTÂNCIA DE NoIdentificador
        CHAMAR ESTE.visitar_identificador(no)
    // ... (Implementar visitantes para todos os tipos de nó da AST)
    SENA
        // Nó desconhecido ou não implementado
    FIM SE
FIM METODO

// Visita um bloco de código (novo escopo)
METODO visitar_bloco(no_bloco)
    CHAMAR ESTE.TABELA_SIMBOLOS.entrar_escopo()

    PARA CADA DECLARACAO EM no_bloco.DECLARACOES
        CHAMAR ESTE.visitar(DECLARACAO)
    FIM PARA

    CHAMAR ESTE.TABELA_SIMBOLOS.sair_escopo()
FIM METODO

// -----
// VISITORS: VERIFICAÇÃO DE DECLARAÇÃO E TIPOS

```

```

// -----

METODO visitar_declaracao_variavel(no_decl)
    // 1. Visita o valor inicial para obter seu tipo
    TIPO_EXPRESSAO = CHAMAR ESTE.visitar(no_decl.VALOR_INICIAL) // Este
método deve retornar o tipo inferido

    // 2. Cria o novo símbolo
    NOVO_SIMBOLO = NOVO Simbolo(no_decl.NOME_VARIAVEL.LEXEMA,
TIPO_EXPRESSAO)

    // 3. Define o símbolo no escopo
    CHAMAR ESTE.TABELA_SIMBOLOS.definir_simbolo(NOVO_SIMBOLO.NOME,
NOVO_SIMBOLO)

    // 4. Armazena o tipo no próprio nó da AST (para uso posterior, como
geração de código)
    no_decl.TIPO_DO_NO = TIPO_EXPRESSAO

    RETORNA TIPO_EXPRESSAO // Retorna o tipo da declaração
FIM METODO

METODO visitar_identificador(no_id)
    // 1. Verifica se a variável/função existe no escopo
    SIMBOLO = CHAMAR ESTE.TABELA_SIMBOLOS.obter_simbolo(no_id.TOKEN.LEXEMA)

    SE SIMBOLO É NULO
        IMPRIMIR_ERRO("Erro Semântico: Variável '" + no_id.TOKEN.LEXEMA + "'
não declarada (Linha: " + no_id.TOKEN.LINHA + ")")
        ESTE.TEM_ERROS = VERDADEIRO
        RETORNA TIPO_ERRO // Retorna um tipo especial de erro para evitar
falhas em cascata
    FIM SE

    // 2. Anota o tipo no nó da AST
    no_id.TIPO_DO_NO = SIMBOLO.TIPO

    RETORNA SIMBOLO.TIPO
FIM METODO

METODO visitar_expressao_binaria(no_binaria)
    // 1. Visita os nós filhos para obter seus tipos
    TIPO_ESQUERDA = CHAMAR ESTE.visitar(no_binaria.ESQUERDA)
    TIPO_DIREITA = CHAMAR ESTE.visitar(no_binaria.DIREITA)

    TIPO_RESULTADO = TIPO_ERRO

    // 2. Realiza a Verificação de Tipos
    SE TIPO_ESQUERDA FOR IGUAL A TIPO_ERRO OU TIPO_DIREITA FOR IGUAL A
TIPO_ERRO
        // Ignora o erro se já houve um erro em um dos operandos

```

```

        TIPO_RESULTADO = TIPO_ERRO
    SENA0 SE no_binaria.OPERADOR É OP_ARITMÉTICO ('+', '-', '*', '/')
        SE TIPO_ESQUERDA É NUMÉRICO E TIPO_DIREITA É NUMÉRICO
            TIPO_RESULTADO = TIPO_ESQUERDA // ou o tipo promovido
        SENA0
            IMPRIMIR_ERRO("Erro de Tipo: Operador " +
no_binaria.OPERADOR.LEXEMA + " requer operandos numéricos.")
            ESTE.TEM_ERROS = VERDADEIRO
            TIPO_RESULTADO = TIPO_ERRO
        FIM SE
    SENA0 SE no_binaria.OPERADOR É OP_RELACIONAL ('==', '!=', '>', '<')
        // ... (Lógica de comparação)
        TIPO_RESULTADO = TIPO_BOOLEANO
        // ... (Lógica para outros operadores)
    FIM SE

    // 3. Anota o tipo no nó
    no_binaria.TIPO_DO_NO = TIPO_RESULTADO

    RETORNA TIPO_RESULTADO
FIM METODO
FIM CLASSE

```

6. TABELA DE SÍMBOLOS

A Tabela de Símbolos funciona como o repositório dinâmico de todos os metadados do código. Ela armazena informações essenciais sobre cada identificador, como variáveis, classes, métodos e canais de comunicação, incluindo seu tipo, escopo e local de definição. Sua função primordial é fornecer o contexto necessário para que o Analisador Semântico realize todas as checagens de coerência de tipos e validações de escopo. Essencialmente, ela é a base de conhecimento que mapeia a estrutura lógica e contextual do projeto.

Exemplo de pseudocódigo de uma Tabela de Símbolos em um Interpretador orientado a objetos:

```
CLASSE TabelaDeSimbolos
    // Estrutura para manter o escopo atual (Lista de Dicionários ou Pilha de Dicionários)
    PILHA_DE_ESCOPOS: Pilha<Dicionario<String, Simbolo>> // Cada Dicionário representa um escopo

    // Construtor
    CONSTRUTOR()
        ESTE.PILHA_DE_ESCOPOS = NOVA Pilha<Dicionario<String, Simbolo>>()
        CHAMAR ESTE.entrar_escopo() // Escopo Global
    FIM CONSTRUTOR

    // Inicia um novo escopo (Ex: entrada em uma função ou bloco {})
    METODO entrar_escopo()
        CHAMAR ESTE.PILHA_DE_ESCOPOS.empilhar(NOVO Dicionario<String, Simbolo>())
    FIM METODO

    // Sai do escopo atual
    METODO sair_escopo()
        SE ESTE.PILHA_DE_ESCOPOS ESTÁ VAZIA
            IMPRIMIR_ERRO("Erro de Escopo: Tentativa de sair de um escopo inexistente.")
        SENAÓ
            CHAMAR ESTE.PILHA_DE_ESCOPOS.desempilhar()
        FIM SE
    FIM METODO

    // Adiciona um símbolo (variável, função, etc.) ao escopo atual
    METODO definir_simbolo(nome, simbolo)
        ESCOPO_ATUAL = ESTE.PILHA_DE_ESCOPOS.topo()
        SE ESCOPO_ATUAL CONTÉM nome
```

```

        IMPRIMIR_ERRO("Erro Semântico: Símbolo '" + nome + "' já definido
neste escopo.")
    SENA0
        ESCOPO_ATUAL[nome] = simbolo
    FIM SE
FIM METODO

// Busca um símbolo, começando pelo escopo mais interno (topo da pilha)
METODO obter_simbolo(nome)
    PARA CADA ESCOPO EM ORDEM INVERSA NA ESTE.PILHA_DE_ESCOPOS
        SE ESCOPO CONTÉM nome
            RETORNA ESCOPO[nome]
        FIM SE
    FIM PARA
    RETORNA NULO // Símbolo não encontrado
FIM METODO
FIM CLASSE

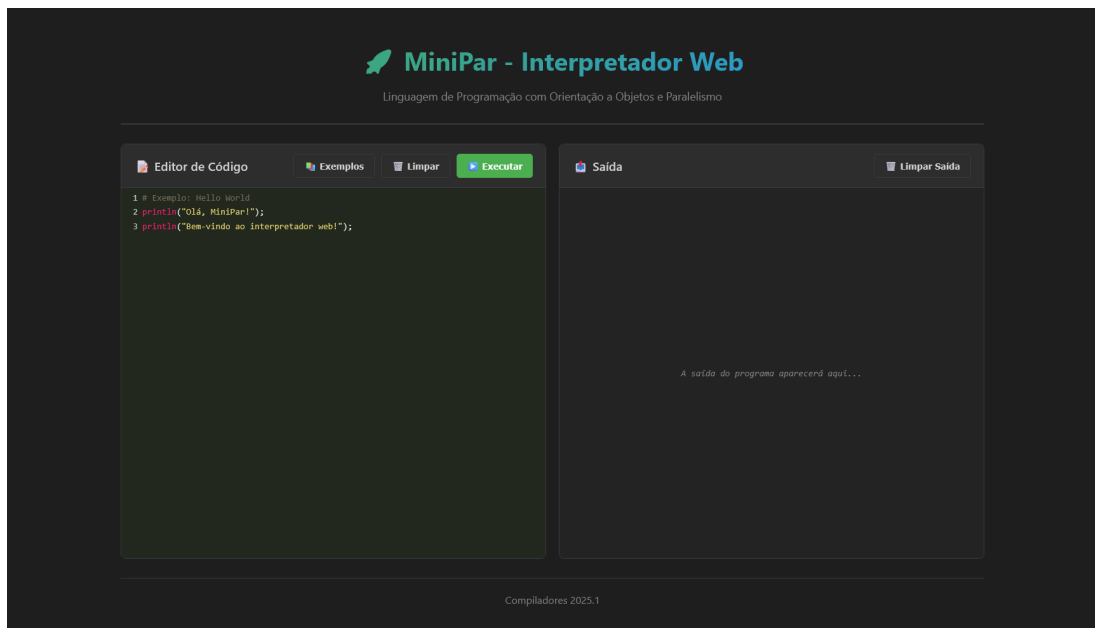
// Classe simples para armazenar informações sobre um símbolo
CLASSE Simbolo
    NOME: String
    TIPO: TipoPrimitivo // Ex: INTEIRO, REAL, BOOLEANO, STRING, FUNCAO
    // ... outros metadados como se é constante, se é parâmetro, etc.
FIM CLASSE

```

7. INTERFACE WEB

Como método adicional de interagir com o interpretador, foi implementada uma interface web leve servida por um servidor HTTP embutido. Essa interface permite:

- Editar código MiniPar com destaque de sintaxe
- Executar o código e visualizar a saída/erros
- Analisar(tokens e AST)
- Operar com sessões de execução interativas (input)



7.1 Endpoints

- GET ``/``
 - Retorna a página ``index.html``
- POST ``/execute``
 - Corpo: texto puro com o código MiniPar
 - Resposta JSON: ``{ "success": boolean, "output": string, "error": string }``
- POST ``/analyze``
 - Corpo: texto puro com o código MiniPar
 - Resposta JSON: ``{ "success": boolean, "tokens": [...], "ast": string, "astTree": object|null, "error": string }``
- POST ``/session/start``
 - Corpo: texto puro com o código MiniPar
 - Resposta JSON: ``{ "sessionId": "uuid" }``

```

- GET `/session/status?sessionId=<id>`
  - Resposta JSON: `{ "running": boolean, "waitingForInput": boolean, "output": string, "error": string }`
- POST `/session/input`
  - Corpo JSON: `{ "sessionId": "...", "input": "linha para stdin" }`
  - Resposta JSON: `{ "success": true }`

```

8. PRODUCT BACKLOG

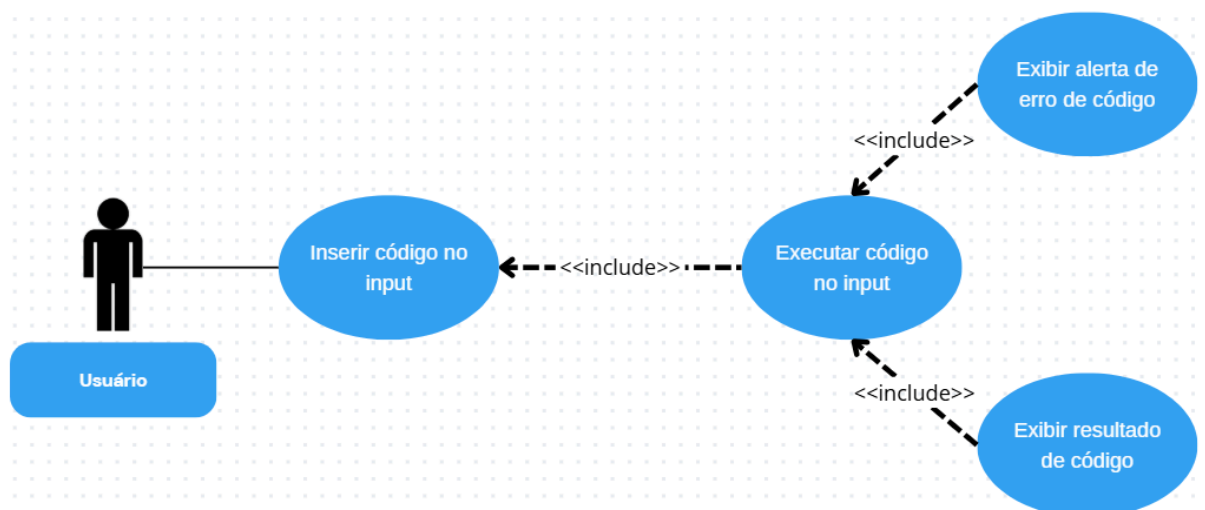
ID	História do Usuário	Prioridade	Observações
1	Definir a gramática da linguagem Minipar para servir de base ao analisador sintático.	Alta	Base do analisador sintático
2	Implementar analisador léxico (lexer)	Alta	Realizar diversos testes
3	Implementar analisador sintático (parser)	Alta	Depende da tarefa 1
4	Implementar analisador semântico	Alta	Precisa da Árvore de Sintaxe
5	Implementar Interface gráfica web	Alta	Conter input para texto e Output
6	Adicionar mensagem de erro com input inválido	Média	Cobrir erros léxicos, sintáticos e semânticos
7	Montar testes do programa e executar no projeto	Alta	Feito através dos pseudocódigos disponíveis no pdf

9. SPRINTS BACKLOG

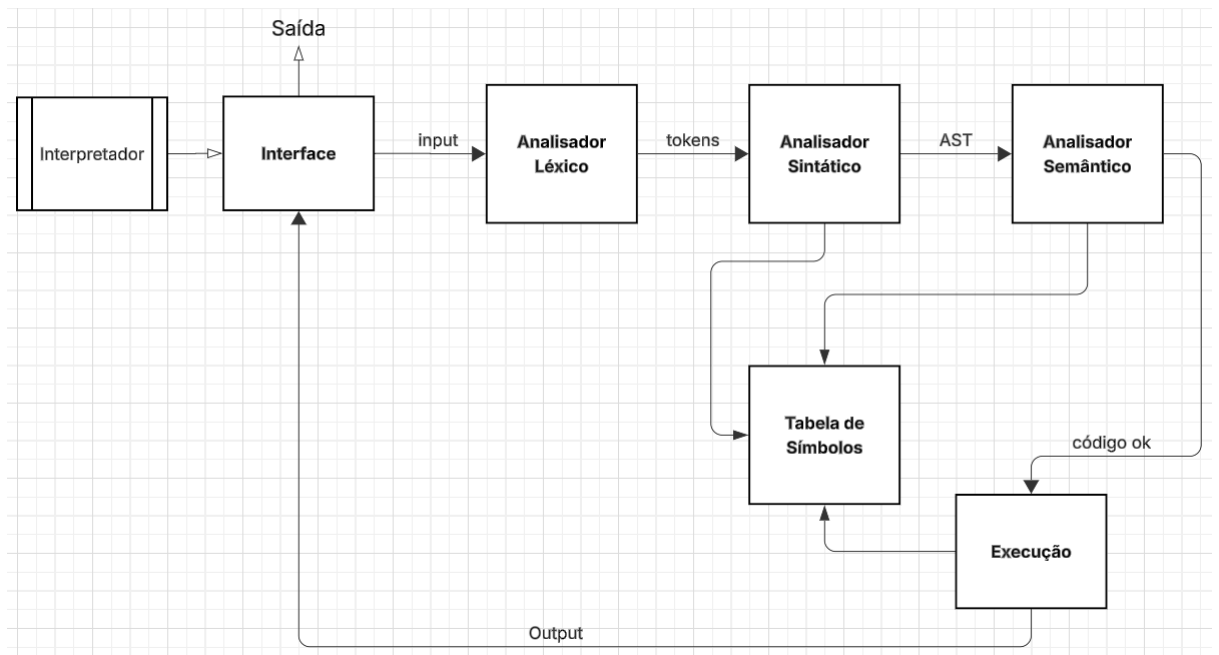
ID	Tarefa	Data Início	Data Final
1	Criar repositório no	07/10/2025	27/10/2025

	GitHub e relatório no docs		
2	Definir a gramática formal da gramática MiniPar	08/10/2025	08/10/2025
3	Implementar Analisador Léxico (Lexer)	08/10/2025	08/10/2025
4	Implementar Analisador Sintático (Parser)	08/10/2025	08/10/2025
5	Implementar Analisador Semântico	08/10/2025	08/10/2025
6	Implementar Interface	28/10/2025	29/10/2025
7	Realizar testes do programa	28/10/2025	29/10/2025
8	Escrever relatório	27/10/2025	29/10/2025

10. DIAGRAMA DE CASOS DE USO (UML)



11. MODELAGEM DE COMPONENTES DE SOFTWARE (UML)



12. DIAGRAMA DE CLASSES (UML)

Classes

```
class Main {
    +main(args: String) void
    +runFile(path: String) void
    +run(source: String, sourceName: String) void
    +printAST(node: ASTNode, depth: int) void
}

class Lexer {
    -source: String
    -tokens: List~Token~
    -start: int
    -current: int
    -line: int
    -column: int
    +scanTokens() List~Token~
    -scanToken() void
    -blockComment() void
    -string() void
    -number() void
}

class Token {
    +type: TokenType
    +lexeme: String
    +line: int
    +column: int
}

class TokenType {
    <<enumeration>>
    IDENTIFIER
    NUMBER
    STRING
}

class Parser {
    -tokens: List~Token~
    -current: int
    +parse() Program
    -declaration() ASTNode
    -classDeclaration() ClassDecl
    -methodDeclaration() MethodDecl
    -functionDeclaration() FuncDecl
    -parameters() List~Parameter~
    -varDeclaration() VarDecl
}
```

```

class Program {
    +statements: List~ASTNode~
    +toString() String
}

class ASTNode {
    <<abstract>>
    +name: String
    +superClass: String
    +attributes: List~VarDecl~
    +methods: List~MethodDecl~
    +returnType: String
    +toString() String
}

```

Relações

Main --> Parser : usa

Main --> Lexer : usa

Parser --> Program : produz AST

Parser --> ASTNode : cria/usa

Parser --> Lexer : consome tokens

Lexer --> Token : produz

Token --> TokenType : tipa

13. TESTES

a. Teste 1: Calculadora Cliente-Servidor

<pre> Eduardo@theduardomaciel MINGW64 ~/Documents/UFAL/Projetos/cl-minipar (main) \$ cd /c/Users/Eduardo/Documents/UFAL/Projetos/cl-minipar && java -cp - ? Análise sintática concluída com sucesso! ?? FASE 3: EXECUÇÃO (INTERPRETER) Digite 1 ou 2: 1 Digite a porta para escutar (ex: 8080): 8080 [TCPChannel calculadora] Servidor escutando na porta 8080 [TCPChannel calculadora] Cliente conectado: /127.0.0.1:62328 Servidor inicializado! [TCPChannel calculadora] Recebido: [*, 12, 7, 0] Recebido: *[TCPChannel calculadora] Enviado: [*, 12, 7, 84.0] </pre>	<pre> Eduardo@theduardomaciel MINGW64 ~/Documents/UFAL/Pro jetos/cl-minipar (main) Canal TCP 'calculadora'] Este processo é (1) Servidor ou (2) Cliente? Digite 1 ou 2: 2 Digite o host do servidor (ex: localhost): localhost [TCPChannel calculadora] Conectado ao servidor localhost:8080 === Calculadora Aritmética === Operações disponíveis: +, -, *, / Digite a operação desejada: * Digite o primeiro valor: 12 Digite o segundo valor: 7 [TCPChannel calculadora] Enviado: [*, 12, 7, 0] [TCPChannel calculadora] Recebido: [*, 12, 7, 84.0] Resultado: 84 </pre>
--	--

b. Teste 2: Execução Paralela: Fatorial e Fibonacci

 **MiniPar - Interpretador Web**
Linguagem de Programação com Orientação a Objetos e Paralelismo

Editor de Código

Exemplos

Limpar

Executar

```
1 # Programa de Teste 2: Execução Paralela
2 # Thread 1: Cálculo do Fatorial
3 # Thread 2: Cálculo da Série de Fibonacci
4
5 class Fatorial {
6     number numero;
7
8     # Construtor
9     Fatorial(number n) {
10         this.numero = n;
11     }
12
13     number calcular() {
14         number resultado = 1;
15         number i = 1;
16
17         while (i <= this.numero) {
18             resultado = resultado * i;
19             i = i + 1;
20         }
21
22         return resultado;
23     }
24
25     void executar() {
26         println("=== Cálculo do Fatorial ===");
27         # number n = input("Digite um número para calcular o fatorial: ");
28         # this.numero = n;
29         number resultado = this.calcular();
30         print("Fatorial de ");
31         print(this.numero);
32         print(" = ");
33         print(resultado);
34         println(" ");
35     }
36 }
37
38 class Fibonacci {
39     number limite;
40
41     # Construtor
42     Fibonacci(number lim) {
43         this.limite = lim;
44     }
45
46     void executar() {
47         println("=== Série de Fibonacci ===");
48         # number n = input("Digite quantos números da série deseja: ");
49         # this.limite = n;
50
51         number a = 0;
52         number b = 1;
53         number i = 0;
54
55         println("Série de Fibonacci: ");
56
57         while (i <= this.limite) {
58             print(a);
59             print(" ");
60
61             number temp = a + b;
62             a = b;
63             b = temp;
64             i = i + 1;
65         }
66     }
67 }
68
69 # Programa principal - Execução Paralela
70 par {
71     # Thread 1: Cálculo do Fatorial
72     Fatorial fatorial = new Fatorial(5);
73     fatorial.executar();
74
75     # Thread 2: Cálculo da Série de Fibonacci
76     Fibonacci fibonacci = new Fibonacci(10);
77     fibonacci.executar();
78 }
```

Resultados

Limpar

Saída

Tokens

Árvore Sintática

```
=== Cálculo do Fatorial ===
Fatorial de 5 = 120
=== Série de Fibonacci ===
Série de Fibonacci:
0 1 1 2 3 5 8 13 21 34 55
```

Compiladores 2025.1

c. Teste 3: Neurônio Simples (Perceptron)

MiniPar - Interpretador Web

Linguagem de Programação com Orientação a Objetos e Paralelismo

Editor de Código

Exemplos

Limpar

Executar

```
1 # Programa de Teste 3: Neurônio Simples
2 # Implementação de um neurônio que aprende usando perceptron
3
4 class Neuronio {
5     number input_val;
6     number output_desejado;
7     number peso;
8     number peso_bias;
9     number taxa_aprendizado;
10
11     # Construtor
12     Neuronio(number inputVal, number output_esperado, number peso_inicial, number
        bias_inicial, number taxa) {
13         this.input_val = inputVal;
14         this.output_desejado = output_esperado;
15         this.peso = peso_inicial;
16         this.peso_bias = bias_inicial;
17         this.taxa_aprendizado = taxa;
18     }
19
20     number ativacao(number soma) {
21         if (soma >= 0) {
22             return 1;
23         } else {
24             return 0;
25         }
26     }
27
28     void treinar() {
29         print("Entrada: ");
30         print(this.input_val);
```

Resultados

Limpar

Saída

Tokens

Árvore Sintática

```
Entrada: 1 | Saída desejada: 0
#### Iteração: 1
Peso: 0,5000
Saída: 1
Erro: -1
Peso do bias: 0,5000
#### Iteração: 2
Peso: 0,4900
Saída: 1
Erro: -1
Peso do bias: 0,4900
#### Iteração: 3
Peso: 0,4800
Saída: 1
Erro: -1
Peso do bias: 0,4800
#### Iteração: 4
Peso: 0,4700
Saída: 1
Erro: -1
Peso do bias: 0,4700
#### Iteração: 5
Peso: 0,4600
Saída: 1
Erro: -1
Peso do bias: 0,4600
#### Iteração: 6
Peso: 0,4500
Saída: 1
Erro: -1
Peso do bias: 0,4500
#### Iteração: 7
Peso: 0,4400
Saída: 1
Erro: -1
Peso do bias: 0,4400
#### Iteração: 8
Peso: 0,4300
Saída: 1
Erro: -1
Peso do bias: 0,4300
#### Iteração: 9
Peso: 0,4200
Saída: 1
Erro: -1
Peso do bias: 0,4200
#### Iteração: 10
Peso: 0,4100
Saída: 1
Erro: -1
Peso do bias: 0,4100
#### Iteração: 11
Peso: 0,4000
Saída: 1
Erro: -1
Peso do bias: 0,4000
#### Iteração: 12
Peso: 0,3900
Saída: 1
Erro: -1
Peso do bias: 0,3900
#### Iteração: 13
Peso: 0,3800
Saída: 1
Erro: -1
Peso do bias: 0,3800
#### Iteração: 14
Peso: 0,3700
Saída: 1
Erro: -1
Peso do bias: 0,3700
#### Iteração: 15
Peso: 0,3600
Saída: 1
Erro: -1
Peso do bias: 0,3600
#### Iteração: 16
Peso: 0,3500
Saída: 1
Erro: -1
Peso do bias: 0,3500
#### Iteração: 17
Peso: 0,3400
Saída: 1
Erro: -1
Peso do bias: 0,3400
#### Iteração: 18
Peso: 0,3300
Saída: 1
Erro: -1
Peso do bias: 0,3300
#### Iteração: 19
Peso: 0,3200
Saída: 1
Erro: -1
Peso do bias: 0,3200
#### Iteração: 20
Peso: 0,3100
Saída: 1
Erro: -1
Peso do bias: 0,3100
#### Iteração: 21
Peso: 0,3000
Saída: 1
Erro: -1
Peso do bias: 0,3000
#### Iteração: 22
Peso: 0,2900
Saída: 1
Erro: -1
Peso do bias: 0,2900
#### Iteração: 23
Peso: 0,2800
Saída: 1
Erro: -1
Peso do bias: 0,2800
#### Iteração: 24
Peso: 0,2700
Saída: 1
Erro: -1
Peso do bias: 0,2700
#### Iteração: 25
Peso: 0,2600
Saída: 1
Erro: -1
Peso do bias: 0,2600
#### Iteração: 26
Peso: 0,2500
Saída: 1
Erro: -1
Peso do bias: 0,2500
#### Iteração: 27
Peso: 0,2400
Saída: 1
Erro: -1
Peso do bias: 0,2400
#### Iteração: 28
Peso: 0,2300
Saída: 1
Erro: -1
Peso do bias: 0,2300
#### Iteração: 29
Peso: 0,2200
Saída: 1
Erro: -1
Peso do bias: 0,2200
#### Iteração: 30
Peso: 0,2100
Saída: 1
Erro: -1
Peso do bias: 0,2100
#### Iteração: 31
Peso: 0,2000
Saída: 1
Erro: -1
Peso do bias: 0,2000
#### Iteração: 32
Peso: 0,1900
Saída: 1
Erro: -1
Peso do bias: 0,1900
#### Iteração: 33
Peso: 0,1800
Saída: 1
Erro: -1
Peso do bias: 0,1800
#### Iteração: 34
Peso: 0,1700
Saída: 1
Erro: -1
Peso do bias: 0,1700
#### Iteração: 35
Peso: 0,1600
Saída: 1
Erro: -1
Peso do bias: 0,1600
#### Iteração: 36
Peso: 0,1500
Saída: 1
Erro: -1
Peso do bias: 0,1500
#### Iteração: 37
Peso: 0,1400
Saída: 1
Erro: -1
Peso do bias: 0,1400
#### Iteração: 38
Peso: 0,1300
Saída: 1
Erro: -1
Peso do bias: 0,1300
#### Iteração: 39
Peso: 0,1200
Saída: 1
Erro: -1
Peso do bias: 0,1200
#### Iteração: 40
Peso: 0,1100
Saída: 1
Erro: -1
Peso do bias: 0,1100
#### Iteração: 41
Peso: 0,1000
Saída: 1
Erro: -1
Peso do bias: 0,1000
#### Iteração: 42
Peso: 0,0900
Saída: 1
Erro: -1
Peso do bias: 0,0900
#### Iteração: 43
Peso: 0,0800
Saída: 1
Erro: -1
Peso do bias: 0,0800
#### Iteração: 44
Peso: 0,0700
Saída: 1
Erro: -1
Peso do bias: 0,0700
#### Iteração: 45
Peso: 0,0600
Saída: 1
Erro: -1
Peso do bias: 0,0600
#### Iteração: 46
Peso: 0,0500
Saída: 1
Erro: -1
Peso do bias: 0,0500
#### Iteração: 47
Peso: 0,0400
Saída: 1
Erro: -1
Peso do bias: 0,0400
#### Iteração: 48
Peso: 0,0300
Saída: 1
Erro: -1
Peso do bias: 0,0300
#### Iteração: 49
Peso: 0,0200
Saída: 1
Erro: -1
Peso do bias: 0,0200
#### Iteração: 50
Peso: 0,0100
Saída: 1
Erro: -1
Peso do bias: 0,0100
#### Iteração: 51
Peso: 0,0000
Saída: 0
Erro: 0
Peso do bias: -0,0000
Parabéns! O neurônio aprendeu.
Valor desejado: 0
```

Compiladores 2025.1

d. Teste 4: Rede Neural para XOR

The screenshot shows the MiniPar Web IDE interface. The left pane, titled 'Editor de Código', contains the following code:

```
1 # Programa de Teste 4: Rede Neural XOR
2 # Implementação de Rede Neural para aprender a função XOR
3 # com uma camada oculta de 3 neurônios e backpropagation
4
5 # Funções auxiliares
6 func sigmoid(number x) -> number {
7     # sigmoid(x) = 1 / (1 + e^(-x))
8     number exp_neg_x = exp(-x);
9     return 1 / (1 + exp_neg_x);
10 }
11
12 func sigmoid_derivative(number x) -> number {
13     return x * (1 - x);
14 }
15
16 class Neuronio {
17     list pesos;
18     number bias;
19     number saida;
20
21     # Construtor
22     Neuronio(number num_inputs) {
23         this.pesos = [];
24         this.saida = 0;
25
26         # Inicializar pesos aleatórios usando built-in random()
27         number i = 0;
28         while (i < num_inputs) {
29             this.pesos[i] = random();
30             i = i + 1;
31         }
32     }
33 }
```

The right pane, titled 'Resultados', shows the output of the program:

```
=== Rede Neural XOR - MiniPar 2025.1 ===

=== Treinando Rede Neural XOR ===
Épocas: 20000
Treinamento concluído!

=== Testando Rede Neural ===
Input: [0, 0], Predicted Output: 0,0147
Input: [0, 1], Predicted Output: 0,9884
Input: [1, 0], Predicted Output: 0,9772
Input: [1, 1], Predicted Output: 0,0218
```

e. Teste 5: Sistema de Recomendação E-commerce

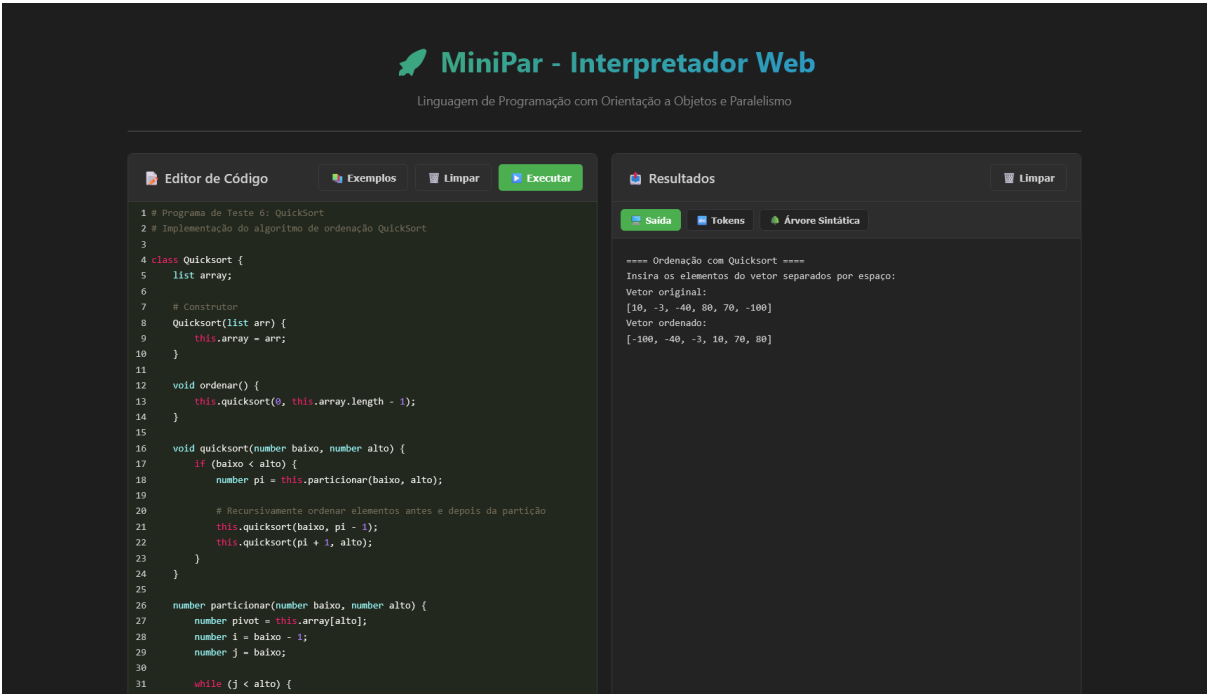
The screenshot shows the MiniPar Web IDE interface. The left pane, titled 'Editor de Código', contains the following code:

```
1 # Programa de Teste 5: Sistema de Recomendação E-commerce
2 # Sistema de recomendação baseado em histórico de compras usando Redes Neurais
3
4
5 # --- Classes ---
6 class Produto {
7     string nome;
8     Produto(string n) {
9         this.nome = n;
10    }
11 }
12
13 class Categoria {
14     string nome;
15     list produtos;
16     Categoria(string n, list prods) {
17         this.nome = n;
18         this.produtos = prods;
19     }
20 }
21
22 class Usuario {
23     list historico_compras;
24     Usuario(list historico) {
25         this.historico_compras = historico;
26     }
27     list codificar_historico(list todos_produtos) {
28         list codificacao = [];
29         number i = 0;
30         while (i < todos_produtos.length) {
31             if (this.comprou(todos_produtos[i])) {
32                 codificacao[i] = 1;
33             } else {
34                 codificacao[i] = 0;
35             }
36             i = i + 1;
37         }
38     }
39 }
```

The right pane, titled 'Resultados', shows the output of the program:

```
Produtos recomendados para você:
Laptop
Tablet
Fones de ouvido
Camisa
Jaqueta
Sapatos
Geladeira
Máquina de lavar
Ar condicionado
Não-ficção
Ficção científica
Fantasia
```

f. Teste 6: Ordenação QuickSort



15 IMPLEMENTAÇÃO

15.1 Tecnologias Usadas

O projeto foi desenvolvido por meio da linguagem orientada a objetos Java, permitindo a criação de classes para cada componente macro do interpretador, e sua interface gráfica (front-end) feita com HTML, CSS e JavaScript básicos.

Categoria	Tecnologia	Versão	Descrição
Linguagem de Programação	Java	17+	Linguagem principal para implementação do interpretador
Ambiente de Desenvolvimento	IntelliJ IDEA	2024.x	IDE principal para desenvolvimento
Ambiente de Desenvolvimento	Visual Studio Code	1.x	Editor alternativo com suporte a Java
Servidor Web	HTTP Server (Java)	Nativo	Servidor HTTP embutido para interface web
Front-end	HTML5	-	Estrutura da

			interface web
Front-end	CSS3	-	Estilização da interface web
Front-end	JavaScript (ES6+)	-	Lógica da aplicação web
Front-end	CodeMirror	5.x	Editor de código com syntax highlighting
Comunicação	Java Sockets	Nativo	Implementação de canais de comunicação (<code>c_channel</code>)
Paralelismo	Java Threads	Nativo	Implementação de blocos <code>PAR</code>
Controle de Versão	Git	2.x	Sistema de controle de versão
Hospedagem	GitHub	-	Repositório remoto e documentação
Modelagem UML	PlantUML / Draw.io	-	Criação de diagramas UML
Build Tool	javac	JDK 17+	Compilador Java nativo
Sistema Operacional	Windows	10/11	Ambiente principal de desenvolvimento
Sistema Operacional	Linux	Ubuntu/Debian	Ambiente de testes
Terminal	Git Bash	2.x	Shell para execução de scripts
Terminal	PowerShell	5.x	Terminal alternativo para Windows
Encoding	UTF-8	-	Codificação de caracteres para suporte multilíngue

15.2 Dependências Principais

Componente	Biblioteca/Framework	Justificativa
Análise Léxica	Regex (Java Pattern)	Reconhecimento de tokens através de expressões regulares
Estruturas de Dados	Java Collections	Listas, Maps e Sets para gerenciamento de tokens, símbolos e AST
Servidor HTTP	<code>com.sun.net.httpserver</code>	Servidor HTTP simples sem dependências externas
I/O	<code>Java NIO</code>	Leitura de arquivos e manipulação de streams
Tratamento de Erros	<code>Java Exceptions</code>	Sistema robusto de exceções customizadas
Execução Paralela	<code>java.util.concurrent</code>	Thread pools e sincronização
Sockets	<code>java.net</code>	Comunicação TCP/IP para canais de comunicação

15.3 Ferramentas de Desenvolvimento

Ferramenta	Propósito
Scripts Batch (.bat)	Automação de compilação e execução no Windows
Scripts Shell (.sh)	Automação de compilação e execução no Linux/Mac
Markdown	Documentação do projeto (README, especificações)
JSON	Configuração de exemplos e dados da aplicação web

15.4 Justificativas Técnicas

- Java 17+:** Escolhido por sua robustez, suporte nativo a threads e sockets, e ampla adoção acadêmica

- b. **Sem Frameworks Externos:** Decisão de usar apenas bibliotecas nativas para simplificar deployment e compreensão
- c. **Interface Web:** Proporciona melhor experiência de usuário com syntax highlighting e execução interativa
- d. **Git/GitHub:** Facilita colaboração em equipe e versionamento do código

15.5 Links Externos

- Link do **repositório no GitHub**: <https://github.com/theduardomacieli/cl-minipar>
- Link do **vídeo de demonstração**: <https://youtu.be/qv40J0rbc4>
- Link das especificações do projeto:
https://drive.google.com/file/d/1JxPJr0AAU5L7Tr06CvkzzrhNk_uVpCOO/view
- Link deste relatório:
https://docs.google.com/document/d/1aPaVa_4Utgj6o9D-JjOhwfNBOsoYYHr5p0_FzE04Lsk/edit?usp=sharing